SmartPark Study Plan (Full Reference)

Here is a full study plan. Work through it in order. Do not skip the concept refresher because Chris will test whether you know the words that describe your code, not just what the code does.

Part One: The Python Building Blocks You Need To Know

Before you read the project files, get comfortable with these five concepts. Every one of them shows up in your code and Chris can ask about any of them.

1. A class is a blueprint, an object is a thing made from that blueprint

A class defines what something looks like and what it can do. An object is one actual example of that class. In your project, `Car` is a class. When a car pulls into the lot the program does `Car("ABC123", "Toyota")` and that gives back an object. The object has a plate, a model, an entry time, and later an exit time.

```python
class Car:
    def __init__(self, plate, model="Unknown"):
        self.plate = plate
        self.model = model
        self.entry_time = datetime.now()
        self.exit_time = None
```

The `__init__` method is the constructor. It runs the moment you create the object. `self` is how the object refers to itself. `self.plate = plate` is saying "save the plate value inside this object under the name plate". After the constructor finishes, the object has those four bits of data stuck to it.

2. Inheritance means one class copies the shape of another

When a class inherits from another class, it automatically has all the methods the parent class has. In your project the `CarPark` class inherits from two parents at the same time.

```python
class CarPark(CarparkSensorListener, CarparkDataProvider):
```

That line says CarPark is a `CarparkSensorListener` and it is also a `CarparkDataProvider`. The two parent classes live in `interfaces.py` and they were written by the lecturer. They are

called abstract base classes (Chris will probably use this exact phrase). An abstract base class is like a contract. It says "if you inherit from me, you must supply these methods".

`CarparkSensorListener` requires you to have `incoming_car`, `outgoing_car`, and `temperature_reading` methods. `CarparkDataProvider` requires you to have `available_spaces`, `temperature`, and `current_time` properties.

That is why your CarPark class has all six of those. If you forgot one, Python would raise an error the moment you tried to make a CarPark object.

3. A property looks like an attribute but runs code

Normally when you write `car.plate` you are reading a value stored in the object. A property is a method that pretends to be a value. You call it without brackets.

```python
@property
def available_spaces(self):
    free = self.total_spaces - len(self.cars)
    if free < 0:
        return 0
    return free
```

Because of the `@property` decorator above the method, you can write `park.available_spaces` and Python secretly runs the method and gives you the result. The nice thing is that the value is always fresh. You never store "number of free bays" as a plain number, so it can never get out of sync with the real list of cars.

4. Reading a JSON file

JSON is just a text file that looks like a Python dict. You open it and use `json.load` to turn it into a real dict.

```python
with open(config_file) as f:
    data = json.load(f)
```

`with open(...) as f:` opens the file and gives it the name `f`. When the `with` block ends, Python closes the file automatically. `json.load(f)` reads the whole file and hands back a dict. After that line `data` is a normal Python dict.

5. Writing to a file in append mode

The `"a"` in `open(self.log_file, "a")` means append. If the file already exists, Python

adds new lines to the end. If it does not exist yet, Python creates it. That is how the audit log grows over the life of the program without wiping itself.

Part Two: Read Each File In This Order

File 1: `smartpark/interfaces.py`

This one you did not write. Chris gave it. But you need to know what is in it. Open the file and read it top to bottom. Notice these things:

- It imports `abc` (short for abstract base class)
- `CarparkSensorListener` has three methods marked with `@abc.abstractmethod`
- `CarparkDataProvider` has three properties marked with `@abc.abstractmethod` and `@property`
- All the method bodies are just `pass`, meaning they do nothing here

Your CarPark class fills in the actual behaviour for all six things.

File 2: `smartpark/car.py`

Very small file. Just holds one class. Read every line and be able to say what it does. In your own words, the Car class is a data holder for one car. It stores the plate, the model, when the car entered, and when the car left.

The `set_exit` method just stamps the current time onto `self.exit_time`. It gets called when the car leaves the lot.

File 3: `smartpark/carpark.py`

This is the biggest file and the one Chris will spend the most time on. Read it three times. First pass, just read for shape. Second pass, understand each method. Third pass, ask yourself what would break if you removed each line.

Section by section.

Imports. `json` and `os` for reading the config, `time` for the localtime function, `datetime` for the log timestamps. `interfaces` gives us the two abstract base classes. `car` gives us the Car class.

Class declaration. `class CarPark(CarparkSensorListener, CarparkDataProvider):`. This is the multiple inheritance line. Chris might ask you why you inherit from two classes. The answer is one parent defines what the sensor sends in (listener), the other defines what the display reads out (data provider).

The `__init__` method. This is what runs when you create a CarPark. Here is the plain English of what it does:

1. If nobody passed a config file path, work out the default path by walking up from this file to the project root and pointing at `samples_and_snippets/config.json`.
2. Save the config file path and the log file path onto the object.
3. Open the config file and read it with `json.load`.
4. If the data has a key called `CarParks`, pull out the first item in that list. Otherwise assume the whole dict is the config.
5. Read the name, location, and total spaces out of the config and save them on the object.
6. Start with an empty list called `self.cars`. This will hold the Car objects that are currently parked.
7. Set the temperature to 22 as a starting value.

The three properties. These come from the data provider contract.

- `available_spaces` calculates free bays by subtracting the length of `self.cars` from `self.total_spaces`. If that goes negative it returns 0 as a safety net.
- `temperature` returns the stored temp as an int.
- `current_time` returns `time.localtime()`.

The three listener methods. These come from the listener contract.

- `incoming_car` first ignores blank plates, then checks the lot is not full, then checks the plate is not already inside, then makes a new Car and adds it to `self.cars`, then logs it.
- `outgoing_car` finds the car by plate, calls `set_exit` on it, removes it from the list, and logs it. If the plate is not in the list it just logs an anomaly without changing anything.
- `temperature_reading` tries to convert the value to a float and saves it. If the conversion fails it just returns without crashing.

The helper method. `write_log` builds a timestamped line and appends it to the log file in append mode.

File 4: `smartpark/config_parser.py`

Tiny helper. It opens a json file and returns the first carpark dict. Nothing tricky. Chris might ask why you have both this and the logic inside CarPark. The honest answer is you kept it around so the tests can use it separately from the CarPark class.

File 5: `smartpark/no_pi.py`

This is the tkinter user interface. You do not need to know tkinter in depth. You need to know the shape.

- `WindowedDisplay` is a helper the lecturer wrote to make the display window. You did not really touch this class. It just builds labels for the fields and updates their text when you call `update`.
- `CarParkDisplay` is your public display. It has three fields: Available bays, Temperature, and At (which is the time). When you construct it, it kicks off a background thread that calls `update_display` once per second. `update_display` reads the three properties off the data provider and pushes the new values into the window. If the bay count is zero it shows the word FULL.
- `CarDetectorWindow` is your fake sensor. It has two buttons and two text boxes. The Incoming Car button calls `incoming_car` on every registered listener. The Outgoing Car button calls `outgoing_car`. Typing in the temperature box calls `temperature_reading` after every keystroke.
- At the very bottom, the `if __name__ == '__main__':` block wires everything together. It builds a tkinter root window, makes a CarPark, makes a display, sets the CarPark as the display's data provider, makes a detector window, adds the CarPark as a listener, then calls `root.mainloop()` which sits and waits for button clicks.

File 6: `tests/carpark_test.py`

Six tests. Each one is a method inside the class `TestCarPark`. Read every method and know what it is checking.

- `test_carpark_reads_config`: makes a CarPark from a small test config, checks total_spaces is 3 and location is Moondalup.
- `test_car_entering_reduces_available_spaces_by_one`: makes a CarPark, calls incoming_car once, checks the count went down by one.
- `test_car_leaving_adds_back_a_space`: makes a CarPark, adds a car, removes it, checks the count went back up.
- `test_available_spaces_never_goes_below_zero`: fills the lot, tries to add more, checks the count stays at zero.
- `test_unknown_car_leaving_does_not_free_a_space`: adds one car, tries to remove a different plate, checks the count did not change.
- `test_temperature_is_updated`: sends a reading of 25, checks the temperature property gives back 25.

The `setUp` method runs before every test. It writes a fresh test config file and creates a CarPark pointing at it. `tearDown` runs after every test and cleans up the temporary files.

File 7: `tests/test_config.py`

One test. It writes a small json string to a file, calls the parser, and checks the parser

returned the right location and total spaces.

Part Three: The Small Change Chris Might Ask For

Chris said the change will be small, something overlooked in the requirements. Practice these before the call, because whatever he asks will probably be close to one of them.

Change one: Add a method that returns the number of cars currently in the lot.

Open carpark.py. Somewhere below `write_log` add:

```python
def car_count(self):
    return len(self.cars)
```

Save. Then open a Python shell or add a quick line to no_pi.py to test it.

Change two: Add a new test.

Open tests/carpark_test.py. Copy an existing test method. Rename it. Change the assertion. For example, a test that checks the temperature stays at the last valid value if you pass a bad reading:

```python
def test_bad_temperature_reading_keeps_last_value(self):
    self.park.temperature_reading(20)
    self.park.temperature_reading("not a number")
    self.assertEqual(self.park.temperature, 20)
```

Change three: Print the carpark name in the log.

Open write_log in carpark.py. Change the line that builds the log line to include `self.name`:

```python
line = stamp + " [" + self.name + "] " + message + "\n"
```

Change four: Show the carpark name in the display window title.

Open no_pi.py. In the main block, change `WindowedDisplay(root, 'Moondalup', ...)` to `WindowedDisplay(root, manager.name, ...)`. But this needs a small chain because CarParkDisplay hard codes 'Moondalup'. You would need to accept a name parameter.

For each of these, practice doing the change, saving the file, then rerunning the tests with `python -m unittest discover -s tests -p "*.py" -v` and showing all tests pass.

Part Four: Questions Chris Is Likely To Ask And How To Answer

"Why did you use inheritance here?" Because the lecturer supplied two abstract classes as the contract between my carpark logic and the display or sensor code. Inheriting means I promise to implement the methods they need, and Python enforces that at construction time.

"Why is available_spaces a property?" Because I want the value to be calculated on the fly from the list of cars, not stored as a separate number. If I stored it separately, the two could drift apart. As a property, every time somebody reads it, Python runs the subtraction fresh, so it is always correct.

"Walk me through what happens when I press Incoming Car." The button in `CarDetectorWindow` runs the `incoming_car` method on itself. That method loops through the listeners it collected earlier and calls `incoming_car` on each one, passing the current plate from the text box. The listener in this case is my CarPark. My `incoming_car` method checks the plate is not blank, checks the lot is not full, checks the plate is not already there, creates a new Car object, adds it to `self.cars`, then writes a line to the log file.

"Why do you store the cars in a list?" Because it is simple and it lets me search by plate with a small loop. If the lot got much bigger I might switch to a dict keyed by plate for faster lookup, but for this size a list is fine.

"How do you stop the count going negative?" Two ways. First the guard at the top of `incoming_car` returns early if the list is already at the total spaces. Second the `available_spaces` property has a safety check that returns 0 if the subtraction somehow ended up negative.

"What happens if the config file is missing?" Python raises `FileNotFoundError` inside `json.load`. I did not add special handling for that because in practice the config sits alongside the code and this was flagged as a low budget proof of concept, but I could wrap the open in a try except and print a helpful message.

Part Five: Day Before The Call

Do all of this the night before, not the day of.

1. Read every file top to bottom out loud. Even if you feel silly. Reading out loud forces you to slow down.
2. Run the tests once. Make sure all seven pass.
3. Run `python no_pi.py` from inside the smartpark folder. Click both buttons a few times. Type a plate. Type a temperature. Watch the display change.
4. Open `carpark_log.txt` and read the lines your test generated.

5. Open GitHub Desktop, click the History tab, and read each commit message and remember roughly what was in it.
6. Do one of the four practice changes above from scratch, without looking at the notes. Save it. Run the tests. Then undo it with git or by deleting the change.

Part Six: On The Call

Have this ready in tabs and windows before you join:

- VS Code open with the project folder loaded
- A terminal window inside VS Code with the project folder as the current directory
- GitHub Desktop open showing the History tab
- Your browser open to the GitHub repo page

When Chris asks you to share, share your whole desktop, not just one window, so switching apps is smooth.

Start by offering to walk him through the code structure. Say something like "I can start with car.py, then carpark.py, then the tests, then no_pi.py if that works". This puts you in a proactive posture rather than waiting for him to grill you.

When he asks for the small change, take a breath, ask him to repeat it if you did not catch it fully, then talk out loud as you do it. Talking out loud is important because it shows him you are thinking, not guessing. Even something like "OK so I need to add a method to the CarPark class, I will open carpark.py and put it near the other helper methods" is worth saying.

After the change, run the tests and show him the pass output. If a test fails, do not panic. Debug it in front of him. Chris will respect that more than pretending everything is fine.

If you get stuck for more than a minute on any question, be honest. Say "I am not sure about that one, can you give me a moment to look at the code". It is better than making something up. Chris can tell the difference.

You have the material you need. Spend two hours tonight reading the code and doing one practice change, and the call will be short and easy.

Added: Quick Reference (for phone, no laptop needed)

This section is new, added on top of the original material above (nothing above was changed).

60 Second Cheat Sheet

Concept	One line answer
Class vs object	Class is the blueprint, object is the actual thing made from it
Inheritance	Child class automatically gets the parent's methods, must fill in any abstract ones
Abstract base class	A contract, Python will not let you build the object unless you supply every required method
Property	A method that runs like a value, so it is always fresh, never stored and forgotten
json.load	Turns a text file that looks like a dict into a real Python dict
Append mode "a"	Adds new lines to the end of a file, creates the file if it does not exist

The Six Contract Methods, In One Breath

From `CarparkSensorListener`: `incoming_car`, `outgoing_car`, `temperature_reading` From `CarparkDataProvider`: `available_spaces`, `temperature`, `current_time`

Test Names, In One Breath

1. reads config
2. incoming car reduces spaces
3. outgoing car frees a space
4. spaces never go below zero
5. unknown plate leaving does nothing
6. temperature reading updates

The Four Practice Changes, Compressed

1. `car_count(self): return len(self.cars)`
2. New test method, copy an existing one, change the assertion
3. Add `self.name` into the `write_log` line
4. Pass the carpark name into `WindowedDisplay` instead of hardcoding 'Moondalup'

Mental Rehearsal Script (read this on the train, no laptop needed)

Say it out loud to yourself, even quietly:

"CarPark inherits from two abstract base classes because one side is the sensor feeding data in, the other side is the display reading data out. Python will not let me construct a CarPark unless I have filled in all six required methods and properties. available_spaces is a property, not a stored number, because I always want it calculated fresh off the actual list of cars, so it can never drift out of sync. When a car comes in, incoming_car checks the plate is not blank, checks the lot is not full, checks the plate is not already parked, then builds a Car object, adds it to the list, and writes a log line. When a car leaves, outgoing_car finds it by plate, stamps the exit time, removes it from the list, and logs it. If Chris asks for a small change, I will talk through what file I am opening and why before I touch anything."

Night Before Checklist (tap through on phone)

- [ ] Read all seven files out loud, top to bottom
- [ ] Ran the test suite, all tests passed
- [ ] Ran no_pi.py, clicked both buttons, typed a plate and a temperature
- [ ] Read carpark_log.txt after a test run
- [ ] Skimmed GitHub Desktop commit history
- [ ] Did one practice change from scratch, then undid it

Day Of Checklist (tap through on phone)

- [ ] VS Code open, project folder loaded
- [ ] Terminal open inside VS Code, correct directory
- [ ] GitHub Desktop open, History tab visible
- [ ] Browser open to the GitHub repo page
- [ ] Ready to share whole desktop, not just one window
- [ ] Opening line ready: "I can start with car.py, then carpark.py, then the tests, then no_pi.py if that works"